TDD Refactor Phase - Improve Quality & Security

Clean up code, apply security best practices, and enhance design whilst keeping all tests green and maintaining GitHub issue compliance.

GitHub Issue Integration

Issue Completion Validation

- **Verify all acceptance criteria met** - Cross-check implementation against GitHub issue requirements
- **Update issue status** - Mark issue as completed or identify remaining work
- **Document design decisions** - Comment on issue with architectural choices made during refactor
- **Link related issues** - Identify technical debt or follow-up issues created during refactoring

Quality Gates

- **Definition of Done adherence** - Ensure all issue checklist items are satisfied
- **Security requirements** - Address any security considerations mentioned in issue
- **Performance criteria** - Meet any performance requirements specified in issue
- **Documentation updates** - Update any documentation referenced in issue

Core Principles

Code Quality Improvements

- **Remove duplication** - Extract common code into reusable methods or classes
- **Improve readability** - Use intention-revealing names and clear structure aligned with issue domain
- **Apply SOLID principles** - Single responsibility, dependency inversion, etc.
- **Simplify complexity** - Break down large methods, reduce cyclomatic complexity

Security Hardening

- **Input validation** - Sanitise and validate all external inputs per issue security requirements
- **Authentication/Authorisation** - Implement proper access controls if specified in issue
- **Data protection** - Encrypt sensitive data, use secure connection strings
- **Error handling** - Avoid information disclosure through exception details
- **Dependency scanning** - Check for vulnerable packages (`npm audit`, `pip audit`, `dotnet list package --vulnerable`, etc.)
- **Secrets management** - Use environment variables or a secrets manager; never hard-code credentials
- **OWASP compliance** - Address security concerns mentioned in issue or related security tickets

Design Excellence

- **Design patterns** - Apply appropriate patterns (Repository, Factory, Strategy, etc.)
- **Dependency injection** - Use DI container or constructor injection for loose coupling
- **Configuration management** - Externalise settings using environment variables or config files
- **Logging and monitoring** - Add structured logging appropriate to your stack for issue troubleshooting
- **Performance optimisation** - Use async/await or equivalent concurrency primitives, efficient collections, caching

Language Best Practices (Polyglot)

- **Null safety** - Enable strict null checks (TypeScript), nullable reference types (C#), or Optional types (Java/Kotlin)
- **Modern language features** - Use pattern matching, destructuring, and idiomatic constructs for your language
- **Memory & performance** - Apply language-specific optimisations only when profiling reveals a bottleneck
- **Error handling** - Use specific error/exception types; avoid swallowing errors silently

Security Checklist

☐ Input validation on all public methods
☐ SQL injection prevention (parameterised queries)

- ☐ XSS protection for web applications
- ☐ Authorisation checks on sensitive operations
- ☐ Secure configuration (no secrets in code)
- ☐ Error handling without information disclosure
- ☐ Dependency vulnerability scanning
- ☐ OWASP Top 10 considerations addressed

Execution Guidelines

1. **Review issue completion** - Ensure GitHub issue acceptance criteria are fully met
2. **Ensure green tests** - All tests must pass before refactoring
3. **Confirm your plan with the user** - Ensure understanding of requirements and edge cases. NEVER start making changes without user confirmation
4. **Small incremental changes** - Refactor in tiny steps, running tests frequently
5. **Apply one improvement at a time** - Focus on single refactoring technique
6. **Run security analysis** - Use static analysis tools (SonarQube, Checkmarx)
7. **Document security decisions** - Add comments for security-critical code
8. **Update issue** - Comment on final implementation and close issue if complete

Refactor Phase Checklist

- ☐ GitHub issue acceptance criteria fully satisfied
- ☐ Code duplication eliminated
- ☐ Names clearly express intent aligned with issue domain
- ☐ Methods have single responsibility
- ☐ Security vulnerabilities addressed per issue requirements
- ☐ Performance considerations applied
- ☐ All tests remain green
- ☐ Code coverage maintained or improved
- ☐ Issue marked as complete or follow-up issues created
- ☐ Documentation updated as specified in issue